INR4torch: A Modular PyTorch Framework for Physics-Informed Neural Networks with Implicit Neural Representations

Peter Naylor*
Φ-lab, ESRIN, ESA, Frascati, Italy

Abstract

We present INR4torch, an open-source Python package that provides a modular and extensible framework for solving partial differential equations (PDEs) using Physics-Informed Neural Networks (PINNs) built on Implicit Neural Representations (INRs). The package supports five neural architectures—SIREN, Random Fourier Features, WIRES, Multiplicative Filter Networks, and Kolmogorov–Arnold Networks—under a unified interface that allows researchers to swap models via configuration without modifying application code. INR4torch includes advanced training techniques such as self-adapting loss balancing, temporal causality weighting, and mixed-precision training, alongside integrated hyperparameter optimization via Optuna. The framework is designed around clear extension points: users define custom PDE losses and data loaders by subclassing two base classes, while the training orchestration is handled automatically. INR4torch is available under the MIT license at `https://github.com/PeterJackNaylor/INR4torch`.

1 Introduction

Physics-Informed Neural Networks [Raissi et al., 2019] have emerged as a powerful paradigm for solving forward and inverse problems governed by partial differential equations. By embedding PDE residuals directly into the loss function, PINNs leverage the approximation capacity of neural networks while respecting known physical laws. A parallel development in deep learning—Implicit Neural Representations (INRs)—has shown that neural networks with carefully chosen activation functions can represent continuous signals with high fidelity [Sitzmann et al., 2020, Tancik et al., 2020]. The intersection of these two fields is particularly promising: INR architectures are well-suited for learning smooth, continuous PDE solutions without grid-based discretization.

Despite the growing interest in PINNs, practitioners face a fragmented landscape of implementations. Existing frameworks such as DeepXDE [Lu et al., 2021], NVIDIA Modulus [NVIDIA, 2023], and NeuralPDE.jl [Zubov et al., 2021] offer varying levels of abstraction and architecture support, but none provide a unified PyTorch interface spanning the full spectrum of modern INR architectures—including the recently proposed Kolmogorov–Arnold Networks (KANs) [Liu et al., 2024]—with integrated advanced training strategies.

INR4torch addresses this gap by providing a modular, YAML-configurable framework where the choice of architecture, training strategy, and PDE formulation are cleanly separated. Researchers can define a new PDE problem by subclassing two Python classes and writing a configuration file, leaving the training loop, optimization, and evaluation to the framework.

*Corresponding author. Email: peter.jack.naylor@gmail.com

2 Statement of Need

Researchers working with PINNs frequently need to (1) experiment with different neural architectures to find the best representation for their PDE, (2) apply advanced training strategies such as loss balancing and causal weighting, and (3) conduct systematic hyperparameter searches. Implementing these from scratch for each project is time-consuming and error-prone.

INR4torch is designed for computational scientists and machine learning researchers who want a production-ready PINN toolkit that is easy to extend with new PDE problems while providing access to state-of-the-art architectures and training techniques out of the box. The package is particularly suited for problems where continuous representations are advantageous, such as inverse problems, surrogate modeling, and data assimilation.

3 Software Architecture

INR4torch is organized around three core abstractions that separate data, model, and training concerns.

3.1 Data Layer: `DataPlaceholder`

The `DataPlaceholder` class provides the interface for data loading and normalization. Users subclass it and override the `setup_data` method to define how raw point clouds are split into input coordinates and target values. The base class handles automatic normalization to $[-1, 1]$, batch iteration, and optional GPU transfer.

3.2 Model Layer: `INR`

The `INR` class serves as a unified factory for all supported architectures. Given a model name and hyperparameters, it constructs the appropriate network. Five architectures are currently supported:

- **SIREN** [Sitzmann et al., 2020]: Sinusoidal activation functions with principled weight initialization, well-suited for representing signals with fine spatial detail.
- **Random Fourier Features (RFF)** [Tancik et al., 2020]: Fixed random projections that map low-dimensional inputs into a high-frequency feature space before processing by a standard MLP.
- **WIRES** [Saragadam et al., 2023]: Complex-valued Gabor wavelet activations that provide localized frequency representations, effective for signals with spatially varying frequency content.
- **Multiplicative Filter Networks (MFN)** [Fathony et al., 2021]: Learnable Gabor filters that modulate hidden features at each layer using the original input, enabling progressive frequency refinement.
- **Kolmogorov–Arnold Networks (KAN)** [Liu et al., 2024]: B-spline–based learnable activation functions on each network edge, replacing fixed activations with data-adaptive basis expansions.

All architectures expose the same forward interface, allowing users to swap models by changing a single configuration parameter. Optional skip connections and modified MLP variants are also supported.

3.3 Training Layer: `DensityEstimator`

The `DensityEstimator` class orchestrates the training loop and serves as the primary extension point for PDE-specific logic. Users subclass it and define named loss methods (e.g., `pde`, `periodicity`, `spatial_gradient`) that compute residuals at collocation points sampled within the domain. The base class provides:

- **Self-adapting loss balancing**: Dynamically adjusts the relative weight of each loss term based on gradient norms, following the approach of McClenny and Braga-Neto [2023]. An exponential moving average with configurable decay smooths weight updates.
- **ReLoBRaLo**: An alternative balancing scheme based on relative loss ratios with random lookback [Bischof and Kraus, 2021].
- **Temporal causality weighting**: For time-dependent PDEs, the temporal domain is partitioned into bins, and exponential decay weights enforce that the network satisfies the PDE at earlier times before later times [Wang et al., 2024].
- **Mixed-precision training**: Automatic FP16/BF16 computation with gradient scaling for memory efficiency on modern GPUs.
- **Learning rate scheduling**: Supports step decay and cosine annealing with warm restarts.
- **Early stopping**: Monitors validation loss and halts training when improvement stalls.

3.4 Differential Operators

INR4torch provides a set of differential operator utilities built on PyTorch's automatic differentiation. These include `gradient`, `hessian`, `laplace`, `divergence`, and `jacobian`, all supporting higher-order derivatives. Users call these within their PDE loss methods to compute residuals symbolically.

3.5 Hyperparameter Optimization

The package integrates with Optuna [Akiba et al., 2019] for automated hyperparameter search. Users define a search space and the framework handles trial management, pruning of unpromising runs, and result aggregation. The YAML configuration system ensures that all hyperparameters—architectural, training, and optimization—are tracked and reproducible.

4 Comparison with Existing Frameworks

Table 1 compares INR4torch with three widely used PINN frameworks across key features.

DeepXDE [Lu et al., 2021] provides broad PDE support across multiple backends (TensorFlow, PyTorch, JAX) with an emphasis on ease of use for standard problems. However, it does not natively support the full range of INR architectures or advanced training strategies such as temporal causality weighting. NVIDIA Modulus [NVIDIA, 2023] offers industrial-scale capabilities with multi-GPU support but is tightly coupled to NVIDIA's ecosystem and does not support KAN or wavelet-based architectures. NeuralPDE.jl [Zubov et al., 2021] leverages Julia's scientific computing strengths and symbolic PDE specification but targets a different language ecosystem. INR4torch occupies a distinct niche: a lightweight, pure-PyTorch framework focused on INR architectures with integrated modern training strategies and hyperparameter optimization.

Table 1: Feature comparison of PINN frameworks. ✓ indicates full support, ∼ partial support, and — no support.

Feature	INR4torch	DeepXDE	Modulus	NeuralPDE.jl
Backend	PyTorch	Multi	PyTorch	Julia
SIREN / RFF	✓	∼	✓	—
WIRES / MFN	✓	—	—	—
KAN support	✓	—	—	—
Self-adapting loss balancing	✓	—	✓	—
Temporal causality	✓	—	∼	—
Mixed-precision training	✓	—	✓	—
Optuna integration	✓	—	—	—
YAML configuration	✓	—	✓	—
Language	Python	Python	Python	Julia
License	MIT	Apache-2.0	Apache-2.0	MIT

5 Example Usage

Defining a new PDE problem in INR4torch requires two subclasses. The following snippet illustrates solving the 1D advection equation $\partial_t u + c\,\partial_x u = 0$:

```
import pinns

class AdvectionData(pinns.DataPlaceholder):
    def setup_data(self, pc):
        return pc[:, :2], pc[:, 2:3] # (t, x) -> u

class AdvectionPDE(pinns.DensityEstimator):
    def pde(self, z, z_hat, weight):
        bs = self.hp.losses["pde"]["bs"]
        x = pinns.gen_uniform(bs, self.device)
        t = pinns.gen_uniform(bs, self.device, start=0, end=1)
        x.requires_grad_(True)
        t.requires_grad_(True)
        u = self.model(x, t)
        du_dt = pinns.gradient(u, t)
        du_dx = pinns.gradient(u, x)
        residue = du_dt + du_dx
        return residue

def data_fn(hp, gpu=False):
    return AdvectionData(train), AdvectionData(test)

model, hp = pinns.train(hp, AdvectionPDE, data_fn, pinns.INR, gpu=True)
```

Switching from RFF to KAN requires only changing `model.name: KAN` in the YAML configuration, with no modifications to the PDE or data code.

6 Testing and Quality Assurance

INR4torch includes a comprehensive test suite with over 40 unit and integration tests covering all modules: model architectures, differential operators, loss functions, normalization, training configuration, and the full training loop. Tests verify correctness of gradients against analytical solutions and check edge cases including NaN handling and early stopping behavior. The package uses continuous integration and supports Python versions 3.8 through 3.12.

7 Conclusion

INR4torch provides a modular, extensible framework for physics-informed learning with implicit neural representations. Its unified architecture interface, advanced training strategies, and clean extension points lower the barrier to experimenting with modern INR architectures for PDE-constrained problems. The package is available at `https://github.com/PeterJackNaylor/INR4torch` under the MIT license.

Acknowledgments

References

Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019.

Rafael Bischof and Michael A Kraus. Multi-objective loss balancing for physics-informed deep learning. *arXiv preprint arXiv:2110.09813*, 2021.

Rizal Fathony, Anit Kumar Sahu, Devin Willmott, and J Zico Kolter. Multiplicative filter networks. In *International Conference on Learning Representations*, 2021.

Ziming Liu, Yixuan Wang, Sachin Vaidya, Fabian Ruehle, James Halverson, Marin Soljačić, Thomas Y Hou, and Max Tegmark. KAN: Kolmogorov–Arnold networks. *arXiv preprint arXiv:2404.19756*, 2024.

Lu Lu, Xuhui Meng, Zhiping Mao, and George Em Karniadakis. DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, 63(1):208–228, 2021.

Levi D McClenny and Ulisses M Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023.

NVIDIA. NVIDIA Modulus: A framework for developing physics-informed machine learning models. `https://developer.nvidia.com/modulus`, 2023. Accessed: 2025.

Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

Vishwanath Saragadam, Daniel LeJeune, Jasper Tan, Guha Balakrishnan, Ashok Veeraraghavan, and Richard G Baraniuk. WIRE: Wavelet implicit neural representations. *arXiv preprint arXiv:2301.05187*, 2023.

Vincent Sitzmann, Julien NP Martel, Alexander W Bergman, David B Lindell, and Gordon Wetzstein. Implicit neural representations with periodic activation functions. In *Advances in Neural Information Processing Systems*, volume 33, pages 7462–7473, 2020.

Matthew Tancik, Pratul P Srinivasan, Ben Mildenhall, Sara Fridovich-Keil, Nithin Raghavan, Utkarsh Singhal, Ravi Ramamoorthi, Jonathan T Barron, and Ren Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *Advances in Neural Information Processing Systems*, volume 33, pages 7537–7547, 2020.

Sifan Wang, Shyam Sankaran, Hanwen Bhatt, and Paris Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024.

Kirill Zubov, Zoe McCarthy, Yingbo Ma, Francesco Calisto, Valerio Pagliarino, Simone Azeglio, Luca Bottero, Emmanuel Luján, Valentin Sulzer, Ashutosh Bharambe, et al. NeuralPDE: Automating physics-informed neural networks (PINNs) with error approximations. *arXiv preprint arXiv:2107.09443*, 2021.